



Universidad Nacional Autónoma de México
Facultad de Ingeniería



Práctica 4

Silverio Martínez Andrés

Lab. Organización y Arquitectura de
Computadoras

Grupo: 8

Semestre 2026-2

Profesor: Jorge Ferat Toscano

Fecha de entrega: 28 de marzo de 2026

Práctica 4

Objetivo

Aprender y usar la librería de módulos LPM (Library of parameterized modules).

Introducción

En el diseño contemporáneo de sistemas digitales, la utilización de bloques funcionales reutilizables y parametrizados es fundamental para optimizar la implementación de operaciones aritméticas. Un ejemplo destacado es la librería LPM (Library of Parameterized Modules), que ofrece componentes como el módulo LPM_ADD_SUB. Esta herramienta permite instanciar sumadores y restadores con anchos de datos ajustables, mejorando la productividad y confiabilidad al reducir el diseño manual de lógica compleja.

Aunado a esto, el multiplexor (MUX) desempeña un rol crítico en el direccionamiento de datos; en su configuración 2 a 1, permite seleccionar entre dos señales de entrada mediante una señal de control. El presente reporte detalla el uso de las herramientas Quartus y ModelSim para integrar estos módulos en un diseño funcional, verificando su comportamiento mediante simulación y análisis de resultados en aplicaciones de unidades aritmético-lógicas (ALU).

Desarrollo

1) Crear un proyecto con nombre practica 4

Paso 1: Seleccionamos la opción de “New Project Wizard”

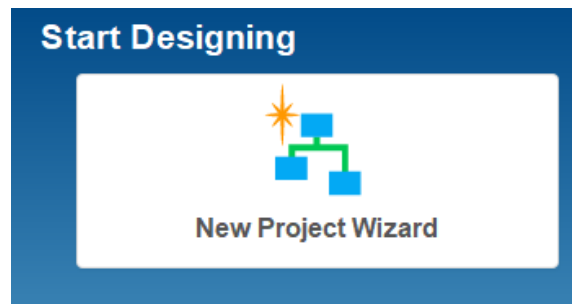


Imagen 1: New Project Wizard

Paso 2: Seleccionamos la carpeta donde queremos guardar el proyecto

Directory, Name, Top-Level Entity

What is the working directory for this project?

C:/Users/imano/Documents/Lab. Arqui/Practica 4

Imagen 2: Directory for this project

Paso 3: Le asignamos el nombre del proyecto

What is the name of this project?

Practica4

What is the name of the top-level design entity for this project? This name is case sensitive and must exactly match the entity name in the design file.

Practica4

Imagen 3: Name of this Practica3

Paso 4:

New Project Wizard

Project Type

Select the type of project to create.

☒ Empty project

Create new project by specifying project files and libraries, target device family and device, and EDA tool settings.

Imagen 4: Project Type

Paso 5: Añadimos el archivo “Practica3”

Add Files

Select the design files you want to include in the project. Click Add All to add all design files in the project directory to the project.
Note: you can always add design files to the project later.

File name: Practica 4

Add

Add All

File Name	Type	Library	Design Entry/Synthesis Tool	HDL Version
-----------	------	---------	-----------------------------	-------------

Remove

Imagen 5: Add Files

Paso 6: Llegamos a esta ventana y ponemos en EP4CE22F17C6

Family & Device Settings

Select the family and device you want to target for compilation.
You can install additional device support with the Install Devices command on the Tools menu.

To determine the version of the Quartus Prime software in which your target device is supported, refer to the [Device Support List](#) webpage.

Device family

Family: Cyclone IV E

Devices: All

Target device

☐ Auto device selected by the Filter

☒ Specific device selected in 'Available devices' list

☐ Other: n/a

Show in 'Available devices' list

Package: Any

Pin count: Any

Core Speed grade: Any

Name filter: Ep4ce22f17c6

☒ Show advanced devices

Available devices:

Name	Core Voltage	LEs	Total I/Os	GPIOs	Memory Bits	Embedded multiplier 9-bit elements	PL
EP4CE22F17C6	1.2V	22320	154	154	608256	132	4

< Back

Next >

Finish

Cancel

Help

Imagen 6: Family & Device Settings

Paso 7: Revisamos que tengamos el simulador ModelSim-Altera y VHDL

EDA Tool Settings

Specify the other EDA tools used with the Quartus Prime software to develop your project.

EDA tools:

Tool Type	Tool Name	Format(s)	Run Tool Automatically
Design Entry/Synthesis	<None>	<None>	☐ Run this tool automatically to synthesize the current design
Simulation	ModelSim-Altera	VHDL	☐ Run gate-level simulation automatically after compilation
Formal Verification	<None>		
Board-Level	Timing	<None>	
	Symbol	<None>	
	Signal Integrity	<None>	
	Boundary Scan	<None>	

Imagen 7: EDA Tool Settings

Paso 8: Para terminar, nos aparecerá la siguiente ventana y le daremos clic en “Finish”

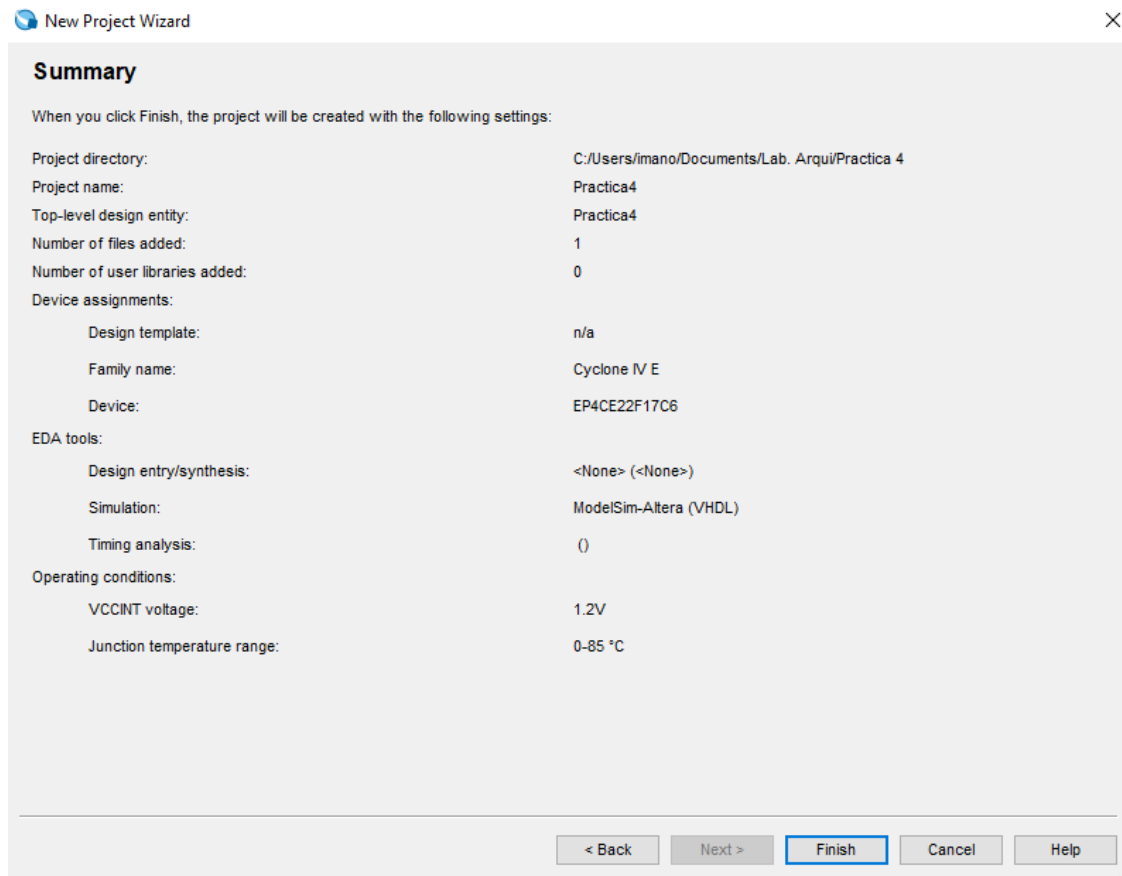


Imagen 8: Summary

2) Generamos por medio del asistente. En la pestaña IP CATALOG Opción librería aritmética

En la pestaña de View, vamos a Utility Windows y después buscamos IP Catalog, damos clic y se nos abrirá una nueva ventana:

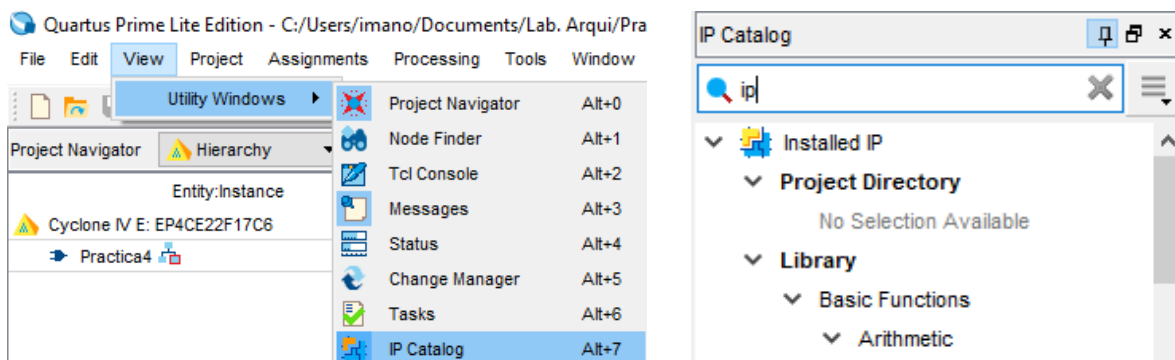


Imagen 9: IP Catalog

3) Seleccionamos la opción LPM_ADD_SUB

EN la ventana de IP Catalog, buscamos en la parte aritmética la opción LPM_ADD_SUB:

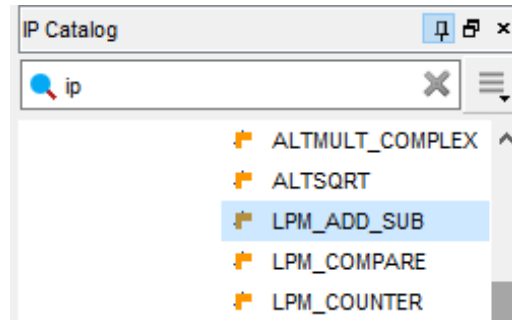


Imagen 10: LPM ADD SUB

Le asignamos el nombre al nuevo archivo:

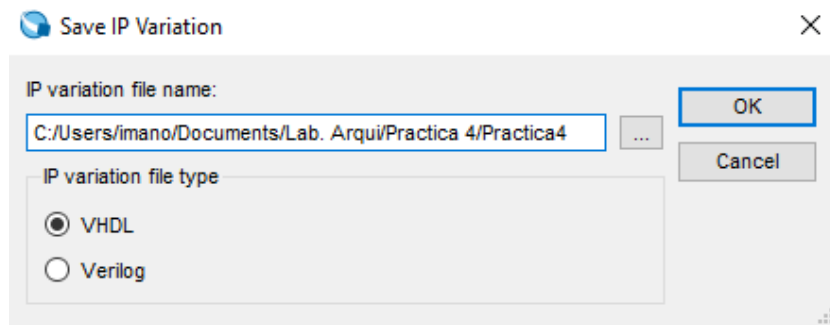


Imagen 11: IP variation file name

4) Se abre el WIZARD para revisar las opciones

Nos aparece la siguiente ventana, donde daremos next:

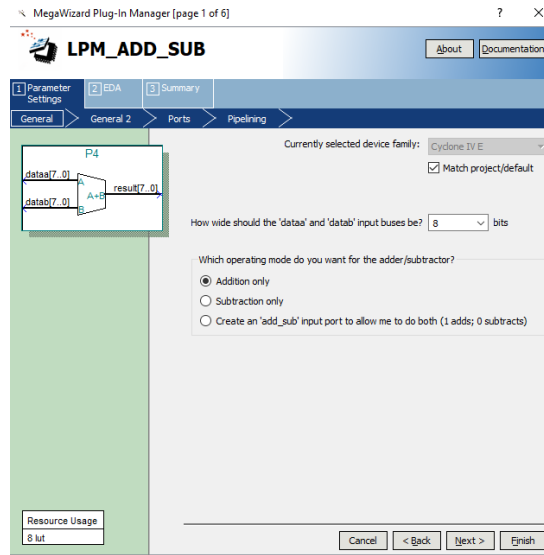


Imagen 12: LPM ADD SUB

5) Tecleamos Next

Nos vuelve aparece una nueva ventana de configuración y volvemos a dar clic en next:

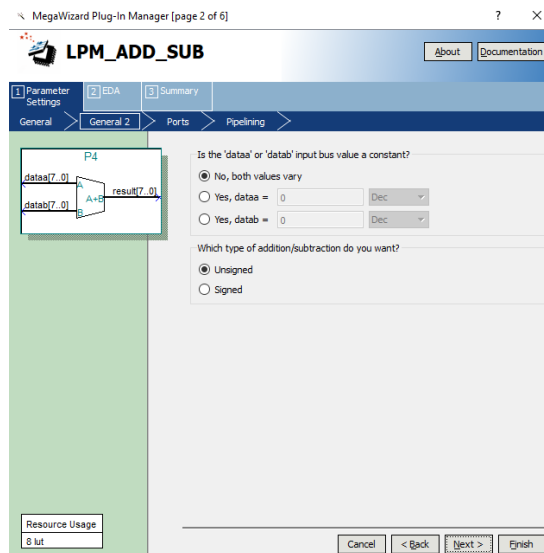


Imagen 13: LPM ADD SUB

6) Opciones de las librerías para este curso sumador y restador (PORTS)

Seleccionamos los puertos necesarios, en este caso, seleccionamos todos los puertos de entrada y salida:

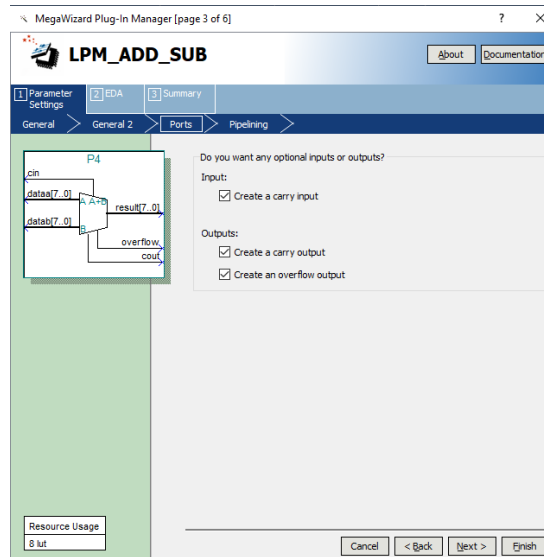


Imagen 14: LPM ADD SUB

7) Continuación de las funciones del sumador – restador (Pipelining)

Nos aparece la siguiente ventana de configuración, donde seleccionamos la opción de NO y damos clic en next:

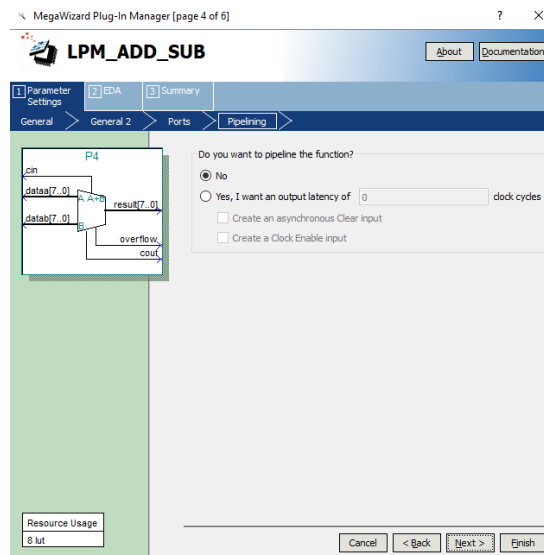


Imagen 15: LPM ADD SUB

8) Resumen

Finalmente nos aparece la siguiente venta de vemos la librería a utilizar en la práctica:

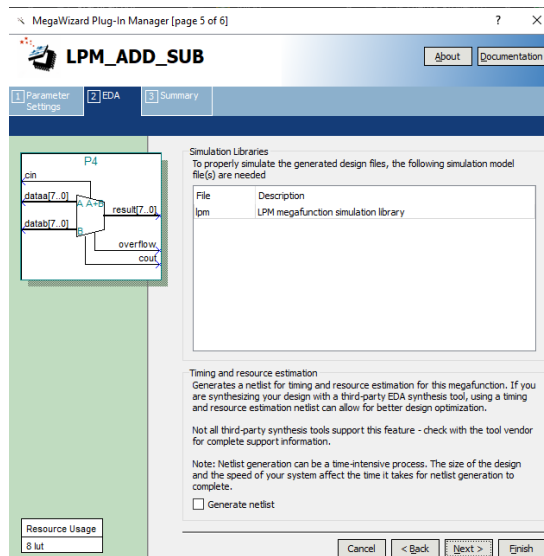


Imagen 16: LPM ADD SUB

Nos aparece en esta ventana, donde observamos los archivos y su descripción, con esta configuración damos clic en next:

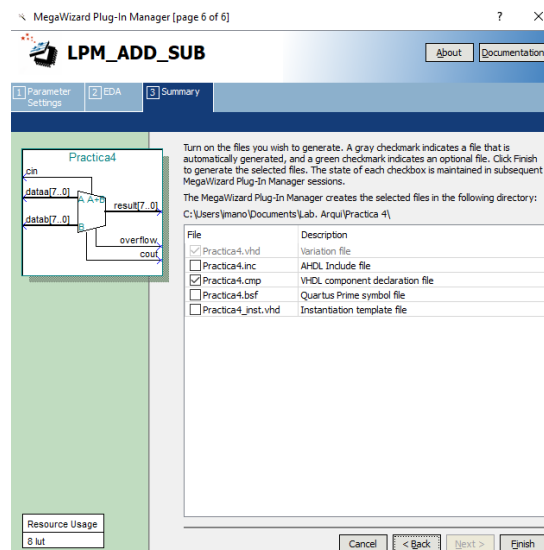


Imagen 17: LPM ADD SUB

Ahora nos aparece la siguiente ventana, donde observamos la ruta donde se aguardara el archivo Practica4.qip:

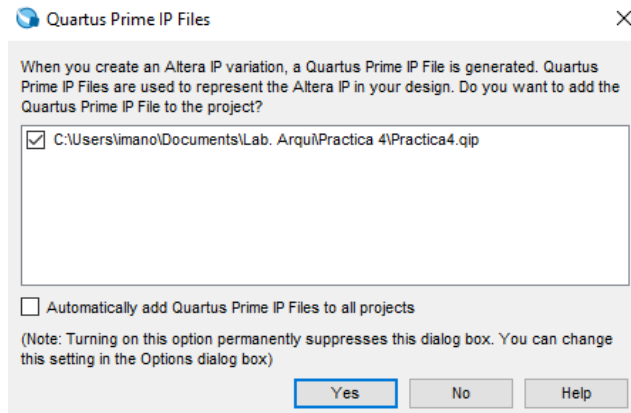


Imagen 18: LPM ADD SUB

Si abrimos el archivo Practica4.qip veremos información de este tipo:

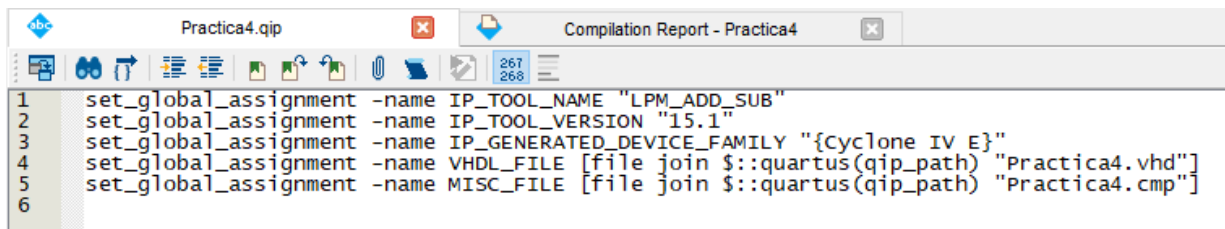


Imagen 19: LPM ADD SUB

Compilamos los archivos anteriormente creados y vemos que se logró compilar de manera exitosa:

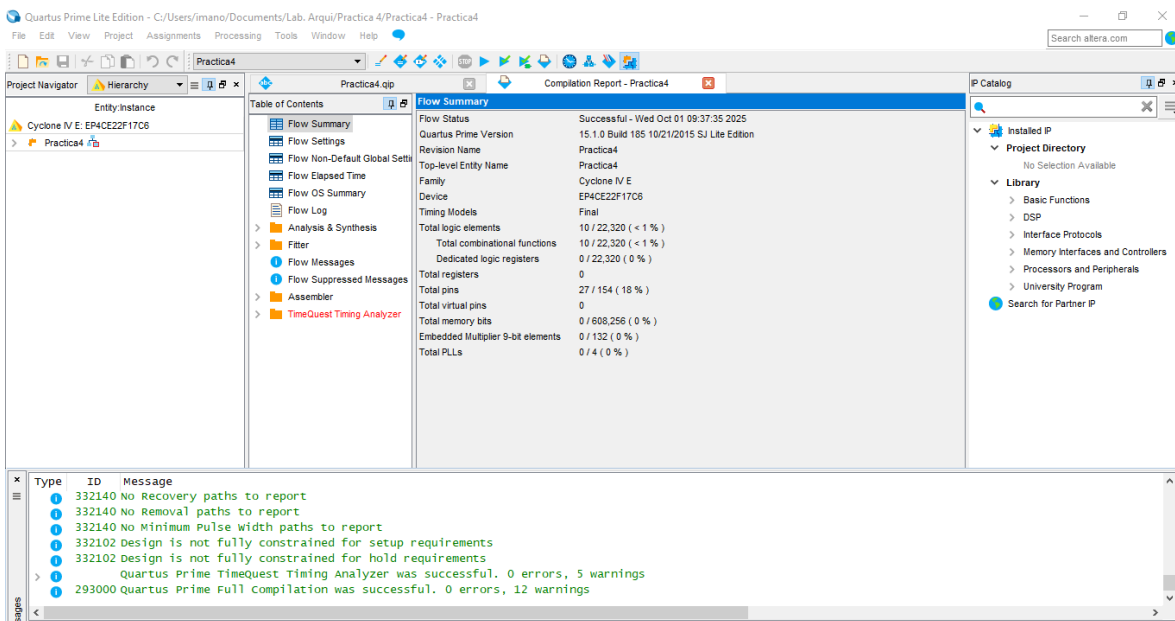


Imagen 20: Compilación

9) Simulación

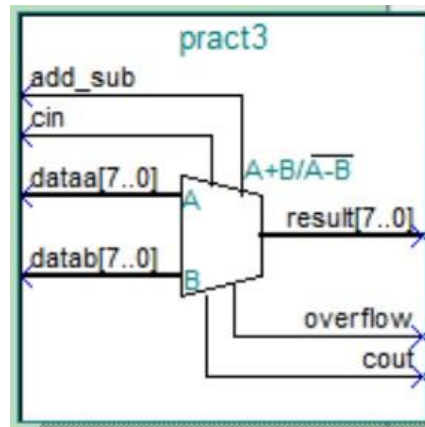


Imagen 21: Circuito

El circuito corresponde a un **bloque de sumador/restador de 8 bits (ALU básica)** con control de operación y banderas de salida.

Entradas

1. **add_sub**
 - Controla la operación:
 - 0 → Suma ($A + B$). 1 → Resta ($A - B$).
 - Configúralo como **valor constante** y cámbialo según quieras probar suma o resta.
2. **cin (Carry In)**
 - Normalmente 0.
 - Puedes ponerlo en 1 para probar sumas con acarreo.
3. **dataa[7..0]** (Entrada A)
 - Vector de 8 bits → configúralo como un **valor binario fijo** o un **patrón de conteo**.
4. **datab[7..0]** (Entrada B)
 - Igual que dataa.

Salidas

1. **result[7..0]**
 - Es el resultado de la operación (suma o resta).
 - Monitorea esta señal para confirmar que coincide con lo esperado.
2. **cout (Carry Out)**
 - Te indica si hubo acarreo en una suma.
3. **overflow**
 - Te indica si hubo desbordamiento en operaciones con signo.

Simulación 1

En la ventana New, en la sección Verification/Debugging Files seleccionar la opción University Program VWF.

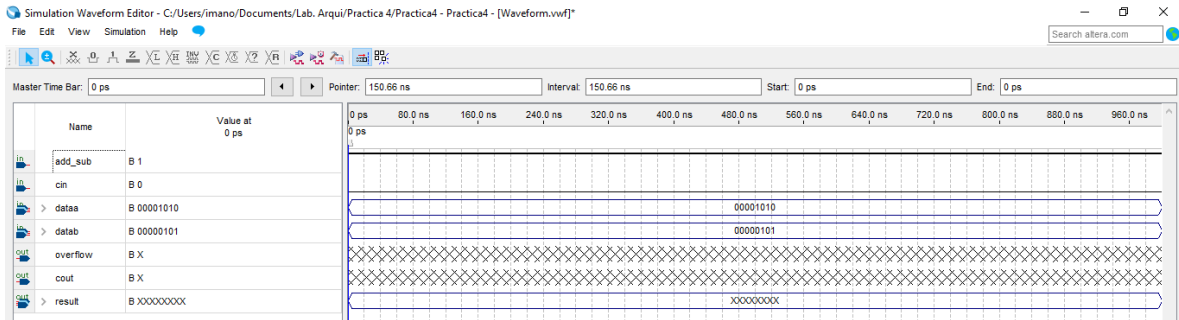


Imagen 22: Simulador

Ejemplo:

Ejemplo de configuración en simulación

- add_sub = 0 (resta).
 - cin = 0.
 - dataa = 11111111 (255 decimal).
 - datab = 00000001 (1 decimal).
- Resultado esperado:**
- result = 00000000 (0 decimal).
 - cout = 1
 - overflow = 1.

Entradas:

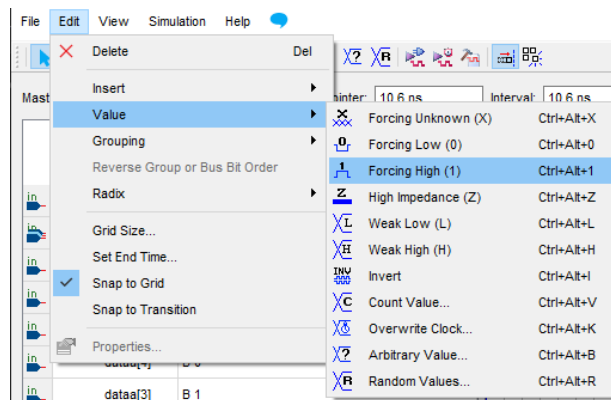


Imagen 23: Entradas

Salida:

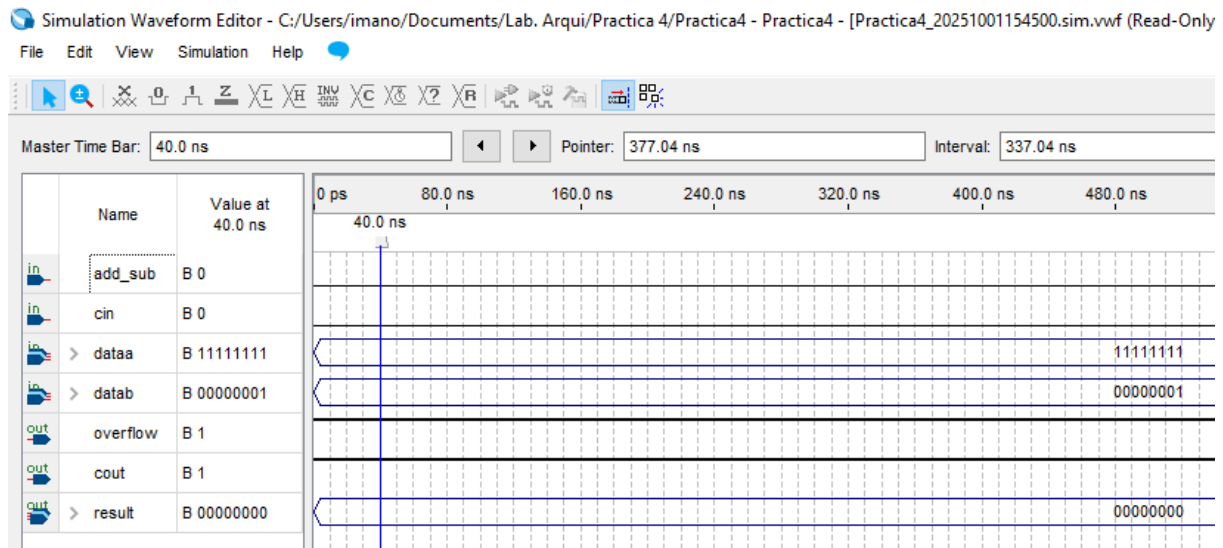


Imagen 24: Resultados

Operación realizada

$$A + B + Cin = 255 + 1 + 0 = 256$$

En **decimal sin signo (unsigned)**, 256 requiere **9 bits** (1 00000000).

En un sistema de 8 bits, el bit adicional se convierte en **cout**.

Resultados esperados

- **result = 00000000 (0 decimal)**
Los 8 bits menos significativos de 1 00000000 son 00000000.
- **cout = 1 (carry out)**
El "1" más significativo se sale de los 8 bits, activando el cout.
- **overflow = 1 (desbordamiento)**
El overflow se interpreta en el contexto de **números con signo** (rango de –128 a +127).
 - 11111111 en binario **con signo** equivale a –1.
 - 00000001 equivale a +1.
 - –1 + 1 = 0 normalmente no tendría problema.
 - Pero, en la lógica del sumador de 8 bits, cuando los operandos tienen **signos distintos** y el resultado no encaja en el rango, se activa el overflow.

En muchos diseños de sumadores, al sumarse 11111111 (−1) + 00000001 (+1) → 00000000 (0), **no debería marcar overflow**, pero algunos RTL implementan $\text{overflow} = (\text{carry_in} \wedge \text{carry_out})$ del bit más significativo, lo que aquí sí da 1.

Interpretación final

Este caso es útil porque muestra dos cosas:

- **cout = 1** → indica que hubo un acarreo fuera del rango de 8 bits (típico en operaciones sin signo).
- **overflow = 1** → según la lógica del circuito, detecta un conflicto en la suma con signo.

En la simulación verás que la salida result se vuelve **00000000**, el cout se activa, y el overflow también aparece encendido.

Simulación 2

Vamos a **configurar entradas y salidas en el simulador** para comprobar su funcionamiento, Abrimos el simulador de ModelSim:

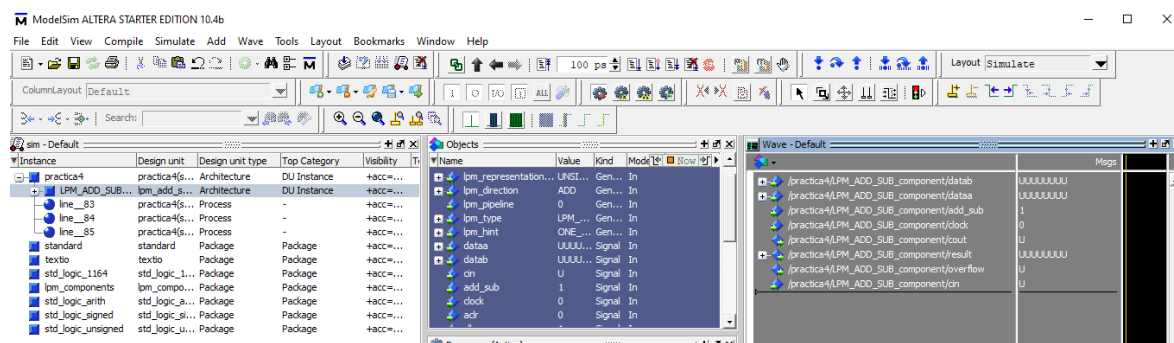


Imagen 25: Simulador

Ejemplo:

Ejemplo de configuración en simulación

- add_sub = 0 (suma).
 - cin = 0.
 - dataa = 00001010 (10 decimal).
 - datab = 00000101 (5 decimal).
- Resultado esperado:**
- result = 00001111 (15 decimal).
 - cout = 0.
 - overflow = 0.

Entradas:

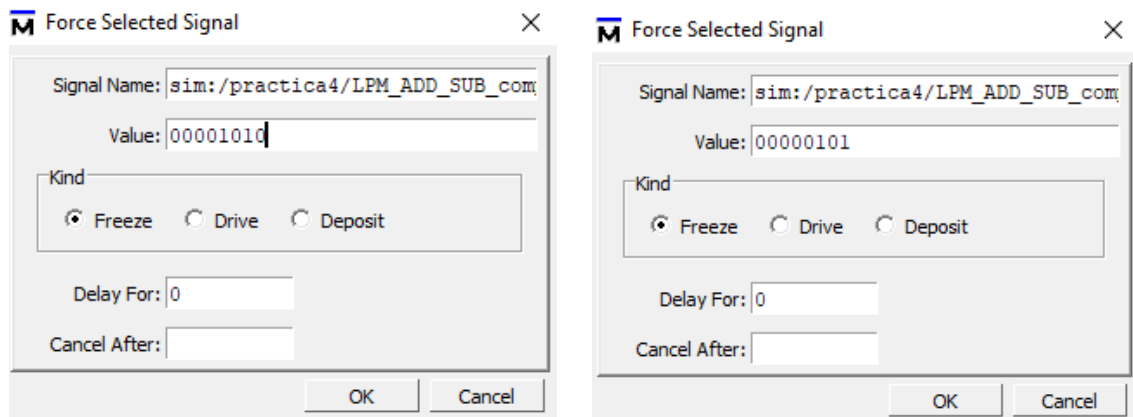
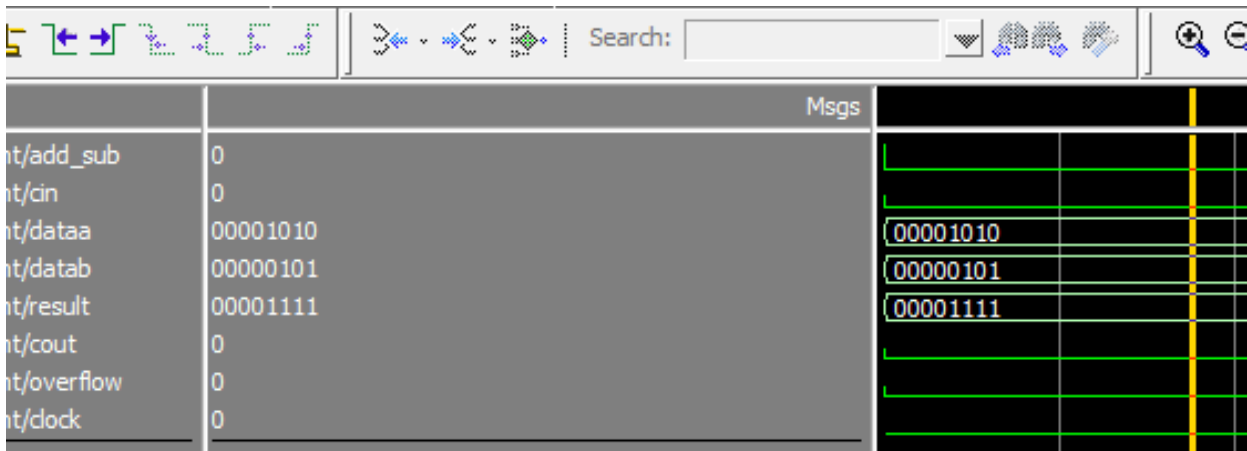


Imagen 26: Entradas

Salidas:



	Msgs
nt/add_sub	0
nt/cin	0
nt/dataa	00001010
nt/datab	00000101
nt/result	00001111
nt/cout	0
nt/overflow	0
nt/clock	0

Imagen 27: Resultados

Operación realizada

La operación es:

$$A + B + Cin = 10 + 5 + 0 = 15$$

Resultados esperados

- **result = 00001111**
 - El número 15 en binario es 1111.
 - Como el bus es de 8 bits, se representa como 00001111.
- **cout = 0 (carry out)**
 - No hubo acarreo fuera del bit más significativo, porque 15 cabe en 8 bits.
- **overflow = 0**

- El desbordamiento ocurre cuando el resultado excede el rango permitido para **números con signo (–128 a +127)**.
- Como 15 está dentro de este rango, no hay overflow.

Interpretación final

Este ejemplo muestra un **caso sencillo de suma sin problemas**:

- Se suman dos números positivos pequeños.
- El resultado cabe en el rango.
- No se activa ninguna condición especial (ni acarreo ni overflow).

En la simulación, verás que la salida result cambia a 00001111, y tanto cout como overflow permanecen en 0.

Conclusión

La implementación exitosa del bloque LPM_ADD_SUB puso de manifiesto las ventajas competitivas que ofrecen las librerías de módulos parametrizados (LPM) al momento de estructurar sistemas aritméticos en dispositivos de lógica programable. Al integrar este componente en un bus de 8 bits dentro del entorno de Quartus, se logró una síntesis eficiente que simplifica el diseño de unidades de suma y resta sin sacrificar el control sobre las señales de estado.

A través de la verificación exhaustiva en ModelSim, se analizaron meticulosamente las transiciones de las banderas de carry-out y overflow, lo que resultó fundamental para comprender cómo el hardware gestiona los límites numéricos. En última instancia, esta práctica no solo validó la precisión del módulo frente a las reglas de la aritmética binaria, sino que también demostró que el uso de bloques pre-optimizados reduce significativamente los tiempos de desarrollo y los posibles errores de diseño manual en arquitecturas de FPGA.

Referencias

1. Intel. (s.f.). *LPM_ADD_SUB: Adder/subtractor megafunction*. Intel. Recuperado el 01 de octubre de 2025 de <https://www.intel.com/content/www/us/en/docs/programmable/683490/20-3/lpm-add-sub-adder-subtractor.html>
2. Adams, V. H. (s.f.). *LPM and ALT_MF User Guide*. Van Hunter Adams. Recuperado el 01 de octubre de 2025 de https://vanhunteradams.com/DE1/Drum/ug_lpm_alt_mfug.pdf

3. GeeksforGeeks. (s.f.). *Multiplexers in Digital Logic*. GeeksforGeeks.
Recuperado el 01 de octubre de 2025 de
<https://www.geeksforgeeks.org/digital-logic/multiplexers-in-digital-logic/>